Exercise 0 - Vector Sum

Kenan Gündogan, Philipp Gündisch

CS3

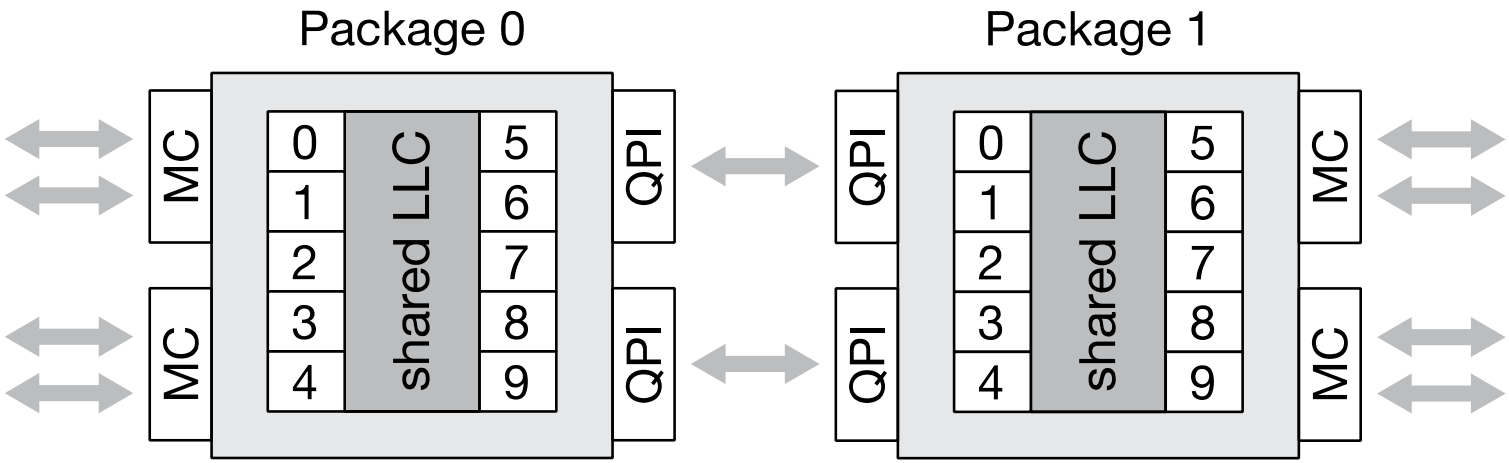
25.10.2023

Cluster resources

- The compute nodes of the RRZE cluster are used in this exercise
  - Do not allocate more resources than necessary
- The Meggie cluster consists of 728 compute nodes
  - 100 GBit/s Intel OmniPath interconnect
  - Dual-socket system with 64 GB of RAM
    - Broadwell *(Intel Xeon E5-2630v4)*
    - 10 cores per chip
    - 2.2 GHz clock frequency
  - More information: https://hpc.fau.de/systems-services/documentation-instructions/clusters/meggie-cluster/

Structure Meggie Node

- Dual-socket Broadwell-EP node
  - 2x Xeon E5-2630 v4 Chips (CPUs, Packages)
  - 10 physical cores and 20 logical cores (SMT-Threads) per chip
  - CPU clock frequency of 2.2 GHz (nominal) and 2.4 GHz peak (turbo) frequency respectively
  - 64 GB RAM

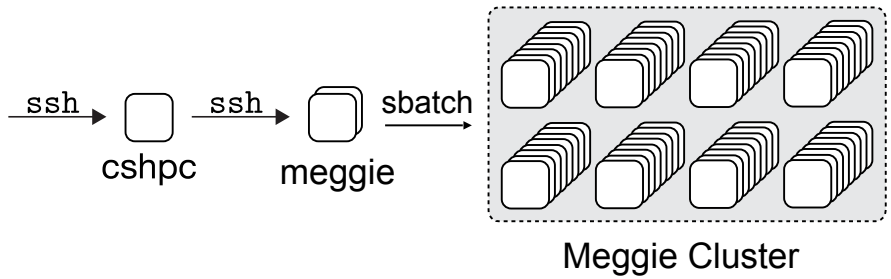


Access Cluster Resources

- For each student an HPC identifier will be assigned in the IdM portal *(hpcvXXXh, etc.)*
- Password must be set initially
  - Log in at https://idm.fau.de
  - Navigate to Profile → Data Overview → Special Services → Service Package HPC → Left on the gear → Change password
  - Note: The old password is your current IdM password
  - The identifier specified in the service package is your cluster login
  - It takes about one hour until the password change is active

Allocate Cluster Node

- Log in to the head node
  - Access from outside of the university network is possible via dialog-server:
    `ssh hpvcXXXh@cshpc.rrze.uni-erlangen.de`
    `ssh meggie.rrze.fau.de`



- Allocate a cluster node for 2 hours:
  `salloc -N 1 --ntasks-per-node=1 --exclusive --cpu-freq=2200000 -t 02:00:00`

`-N 1`	Allocate one node
`--ntasks-per-node 1`	Allocate one core
`--exclusive`	Allocate whole node *(only for measurements)*
`--cpu-freq=2200000`	Set fixed CPU frequency to 2.2 GHz
`-t 02:00:00`	Allocate node for 2 hours

Documentation

- Meggie
  - https://hpc.fau.de/systems-services/documentation-instructions/batch-processing/
  - Measurements via script: `sbatch job_file.sh`
  - `scp get_time.c hpcvXXXh@cshpc.rrze.uni-erlangen.de:`
  - `scp -r ex01/ hpcvXXXh@cshpc.rrze.uni-erlangen.de:`
- GNU Bash
  - https://www.gnu.org/software/bash/manual/bash.html
- GNU Make and Compiler
  - https://www.gnu.org/software/make/manual/make.html
- Commandline Editors
  - Vim
  - Nano
  - Emacs

Exercise 0.1: Using Meggie and provided code

- To list available modules on a meggie node run:
  - `module avail`
- To make `icc` available on a meggie node run:
  - `module load intel`
- Inspect current cluster load
  - https://www.hpc.rrze.fau.de/HPC-Status/emmy.shtml#available-devel-resources
  - To retrieve necessary login data please run `docpw` command on current cluster node
- Select walltime less than 1h: Devel queue (usually shorter waiting times)

Exercise 0.2: Vector Summation

- Implement a function `float vec_sum(float *, uint32_t)` in `src/vec_sum.c` that adds up all elements of an array and returns the calculated sum as return value
- Write a main function in `src/main.c`
  - Command line parameters:
    - Size of array in KiB
    - Measurement time for the benchmark in milliseconds
- Allocate memory for an array dynamically with `malloc`
- Repeatedly call `vec_sum()` until 1 second has elapsed
  - Use `get_time()` for this purpose, which is implemented in the provided file `get_time.c`
- Program output: Processed array elements per second
- Compile the program with the Intel Compiler (`icc`) using the following flags: `-O3 -xHost` *(see Makefile)*

Exercise 0.3: Time Measurement

- Reproducibility of measurement results is very important
- What is a good measurement period for a benchmark?
  - Consider standard deviation of measurement fluctuations
  - Consider turbo boost of the CPU's frequency
- **What measurement time is necessary to retrieve a standard deviation of less than $\pm$ 5% across 20 executions?**

Tasks

- E 0.1: Set Up
  - Log in to Meggie
  - Copy files via `scp`
  - Start an interactive compute-node session (terminate with `exit` or *ctrl+D*)
- E 0.2: Implement Vector Summation
  - Implement on head node
  - Work on compute-node only for benchmarking and testing
  - Implement `src/main.c` and `src/vec_sum.c`
  - Implement `scripts/meggie.sh`
  - Makefile is provided
  - Run benchmark on meggie cluster
  - Save the `result.csv`
- E 0.3: Implement Time Measurement
  - Duplicate `scripts/meggie.sh` and adapt for time measurement
  - Run benchmark on meggie cluster
  - Determine a suitable measurement period